

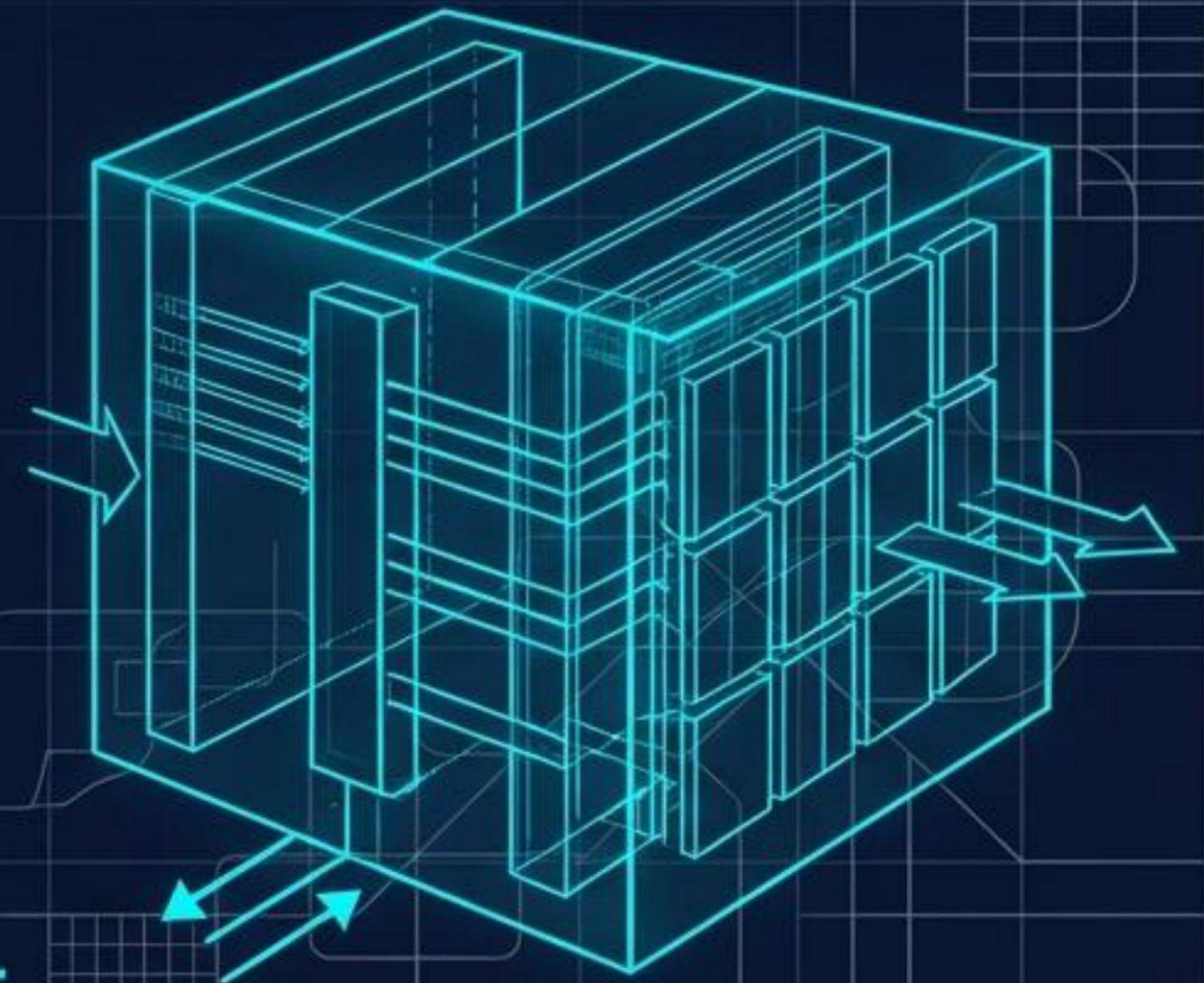
PROYECTO DE COMPARACIÓN DE POLÍTICAS DE REEMPLAZO (LRU, FIFO, LFU)

Para Memoria de Caché Asociativa.

Universidad de Carabobo | Facultad Experimental de
Ciencias y Tecnología | Depto. de Computación

Estudiantes: Jesús Barrera (C.I. 30.281.906)
Edael Duque (C.I. 30800814)

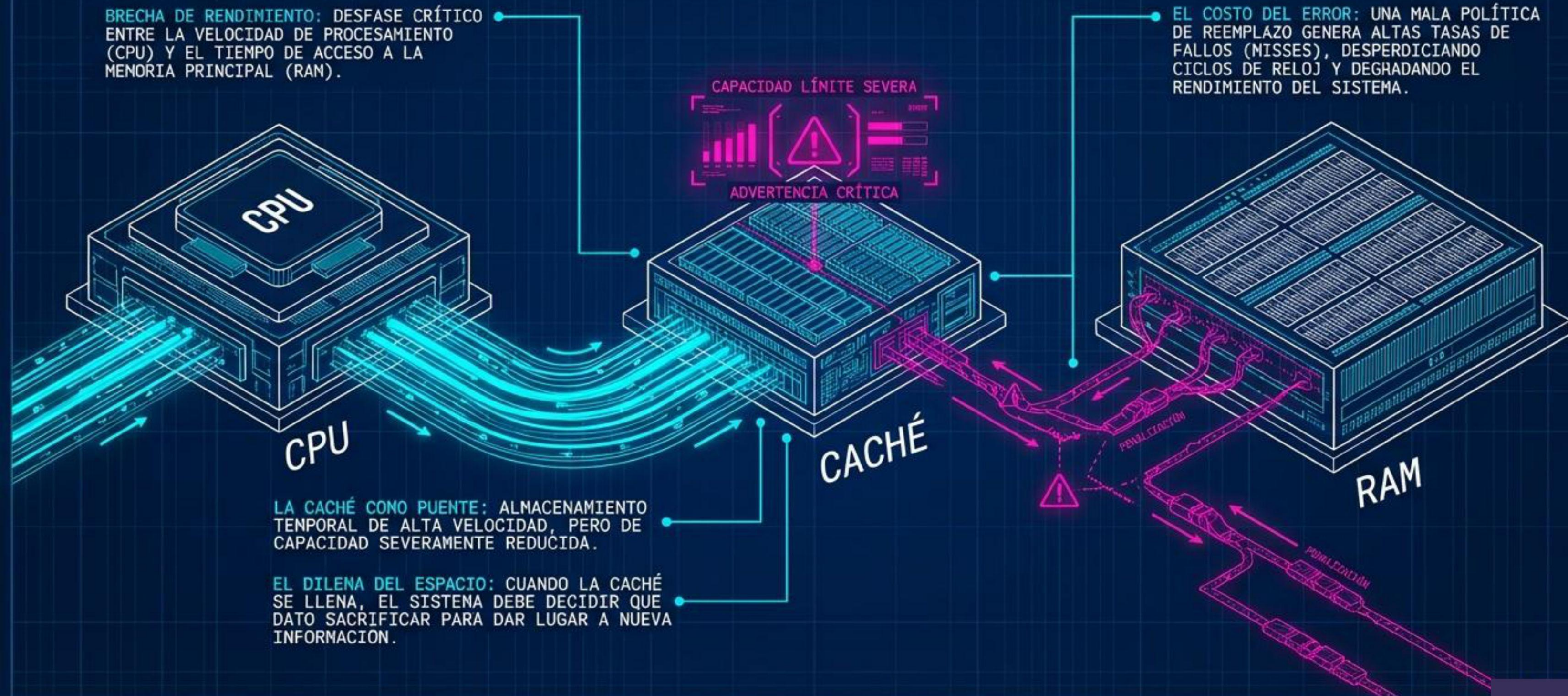
Profesor: José Canache | 5 de marzo de 2026



EL CUELLO DE BOTELLA DE LA LATENCIA

BRECHA DE RENDIMIENTO: DESFASE CRÍTICO ENTRE LA VELOCIDAD DE PROCESAMIENTO (CPU) Y EL TIEMPO DE ACCESO A LA MEMORIA PRINCIPAL (RAM).

EL COSTO DEL ERROR: UNA MALA POLÍTICA DE REEMPLAZO GENERA ALTAS TASAS DE FALLOS (MISSES), DESPERDICIANDO CICLOS DE RELOJ Y DEGRADANDO EL RENDIMIENTO DEL SISTEMA.

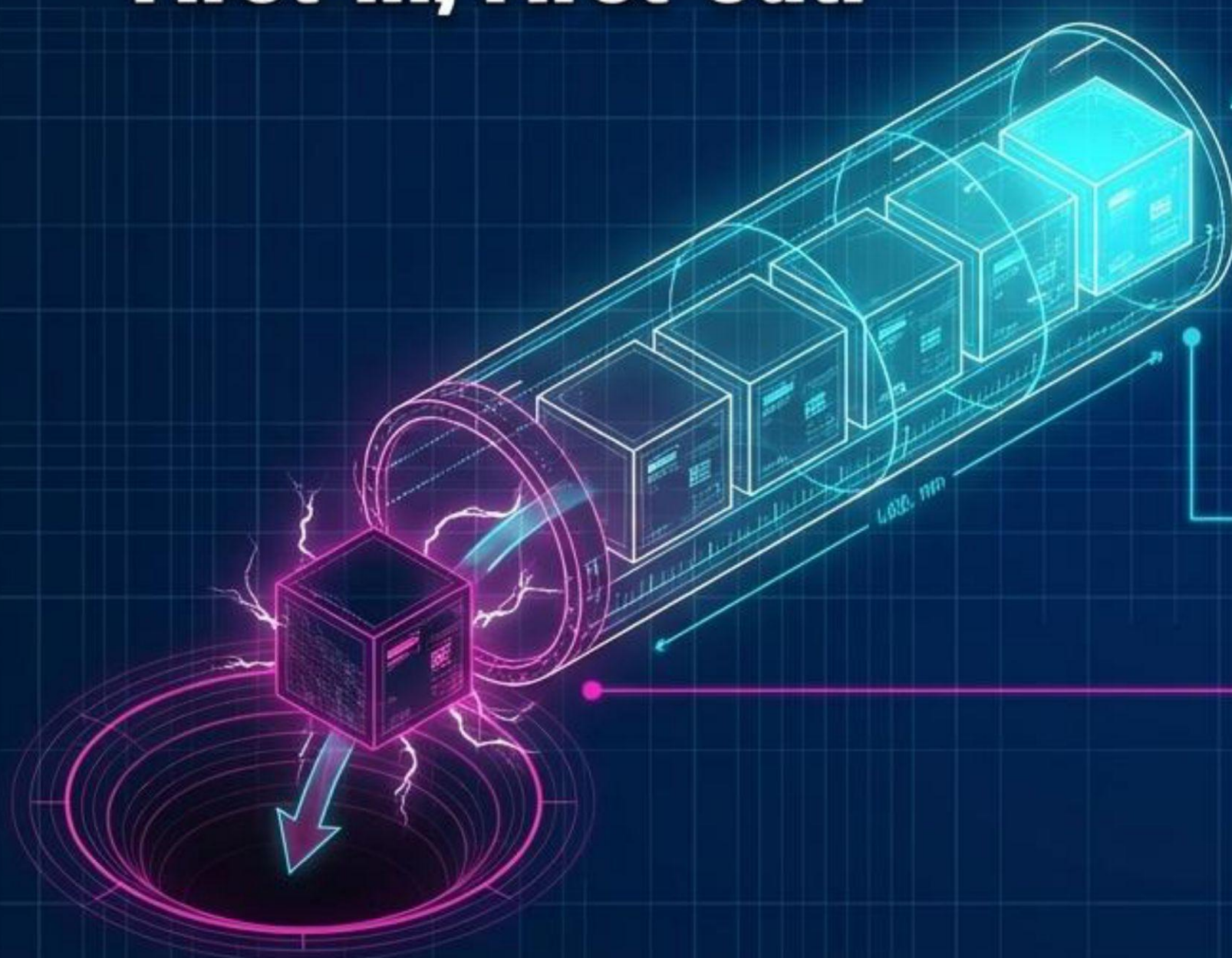


EL OBJETIVO: SIMULACIÓN Y EVALUACIÓN



FIFO: CRONOLOGÍA ESTRICTA.

First-in, First-out.



DIAGNÓSTICO TÉCNICO: POLÍTICA FIFO

• **PRINCIPIO OPERATIVO:** Basado puramente en la antigüedad. El primer bloque que entra es el primero en ser reemplazado.

• **ESTRUCTURA C++:** Implementación mediante colas (Queues).

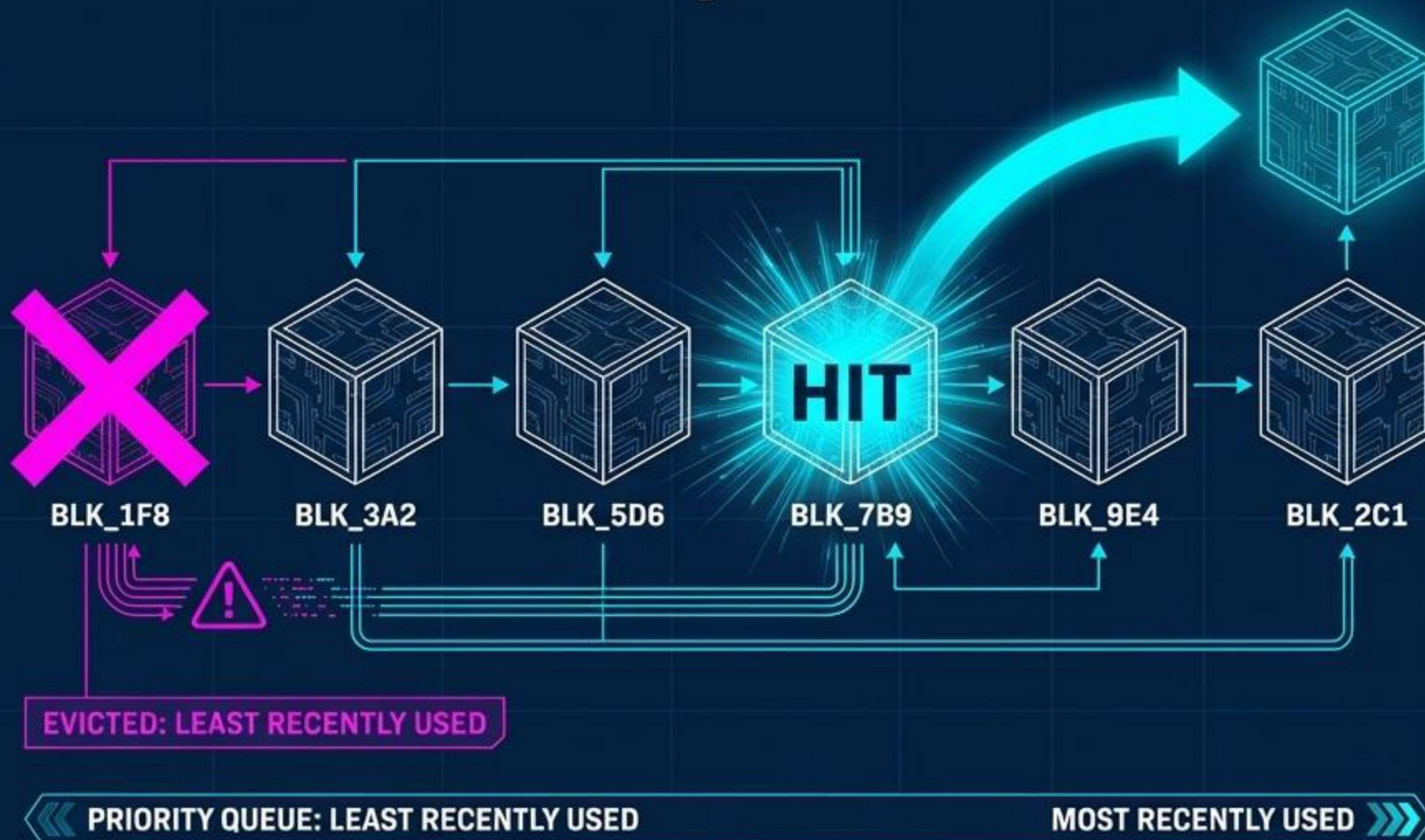
• **VENTAJA:** Muy baja sobrecarga computacional. Procesamiento rápido.

• **RIESGO CRÍTICO:** Ceguera de uso. Puede expulsar datos altamente frecuentes y necesarios, simplemente por haber sido cargados con mayor antigüedad.



LRU: LOCALIDAD TEMPORAL.

Least Recently Used.



DIAGNÓSTICO TÉCNICO: POLÍTICA LRU

PRINCIPIO OPERATIVO:
Expulsa el bloque que ha pasado el mayor periodo de tiempo sin ser utilizado.



LÓGICA PREDICTIVA:
Los datos usados recientemente tienen altísima probabilidad de ser requeridos de nuevo (mayor tasa de aciertos).

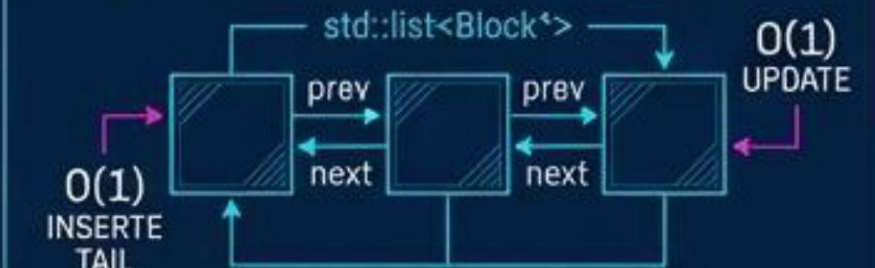


COMPLEJIDAD Y SOBRECARGA:
Exige actualizar el orden de prioridad en cada acceso (sea acierto o fallo).

READ/WRITE ACCESS

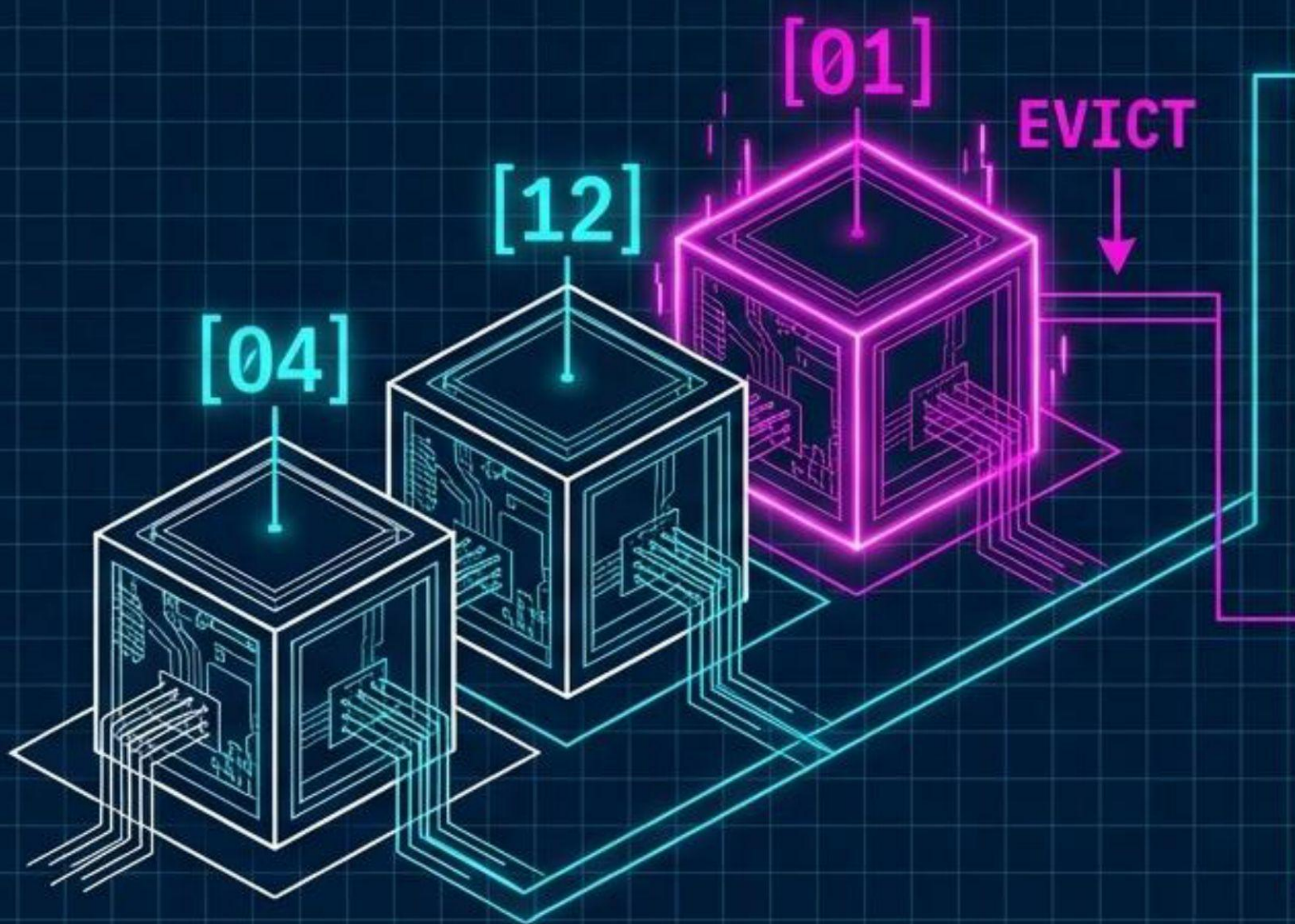
PRIORITY UPDATE

ESTRUCTURA C++:
Requiere al uso de listas doblemente enlazadas para mantener una complejidad de búsqueda lineal eficiente.



LFU: INTENSIDAD DE USO

Least Frequently Used.



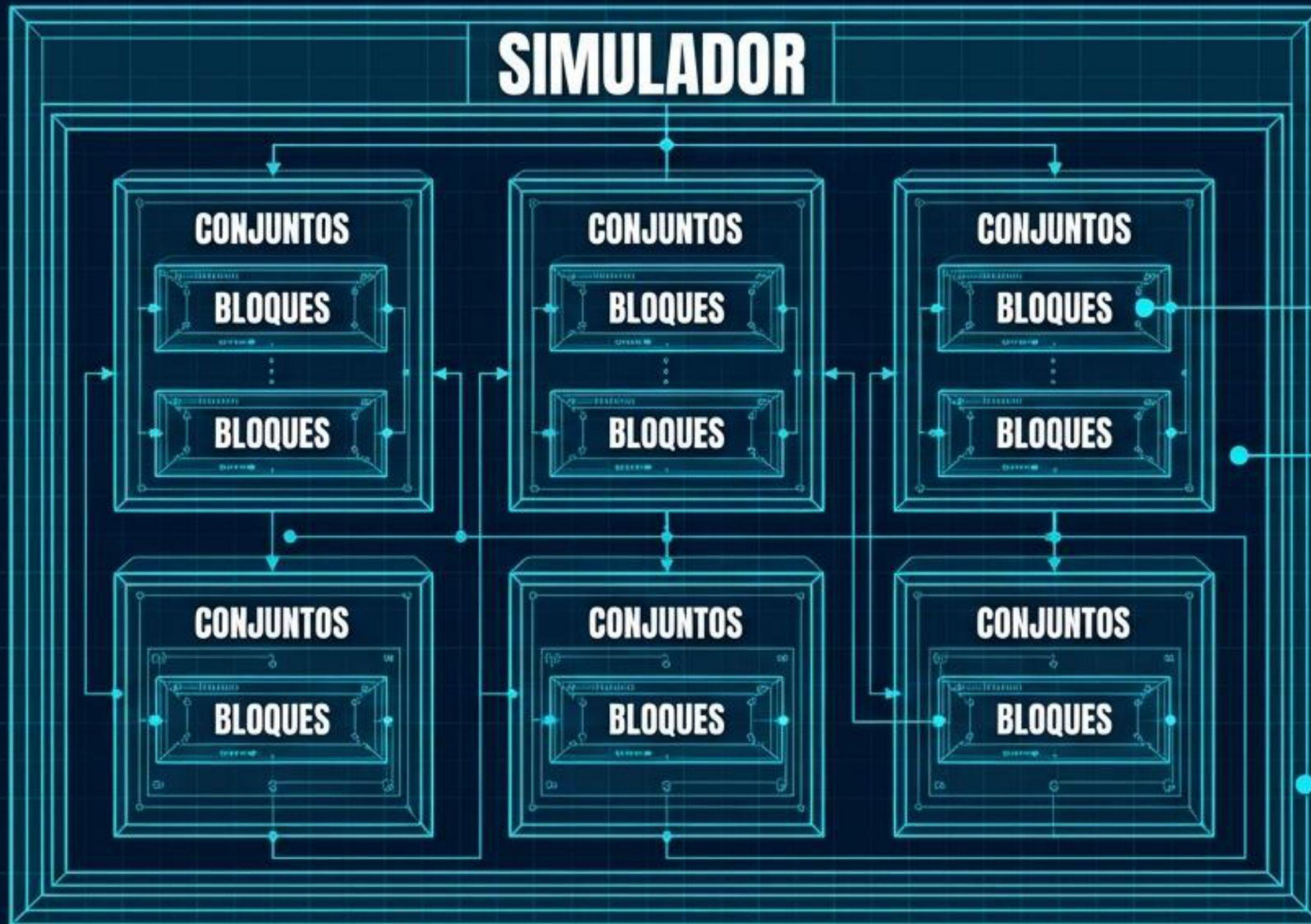
DIAGNÓSTICO TÉCNICO: POLÍTICA LFU

- 🔗 **Principio Operativo:** Rastrea la popularidad. Expulsa el bloque que ha sido solicitado con menor frecuencia desde su ingreso.
- 🔗 **Mecanismo:** Cada bloque posee un contador interno. Un acierto (hit) incrementa este valor. En caso de empate, recurre a FIFO o LRU.
- ⚠️ **Riesgo Crítico: Estancamiento de Caché.** Bloques obsoletos pueden retener contadores altísimos de un uso pasado intensivo, atrincherándose en la memoria e impidiendo la entrada de datos nuevos más útiles.

Matriz Diagnóstica de Políticas.

Política	Criterio de Reemplazo	Sobrecarga Estructural	Vulnerabilidad Principal
FIFO	Cronológico (Antigüedad)	Muy Baja (Colas)	Ignora frecuencia temporal.
LRU	Temporalidad (Último uso)	Media-Alta (Listas doblemente enlazadas)	Penalización de actualización constante.
LFU	Frecuencia (Contadores)	Alta	Estancamiento de caché por popularidad histórica.

ARQUITECTURA C++: DE LÓGICO A FÍSICO.



1. **Bloque (La Unidad Mínima):** Contiene bit de validez, dirección de memoria almacenada, timestamp (para LRU/FIFO) y contador de accesos (para LFU).

2. **Conjunto (El Agrupador):** Implementa la asociatividad. Un vector que agrupa múltiples bloques que comparten el mismo índice dentro de la caché.

3. **Simulador (El Controlador Principal):** Gestiona el vector de conjuntos. Ejecuta la lógica de búsqueda, decreta los Hits/Misses, aplica las políticas y registra las estadísticas globales.

Parámetros del Entorno de Simulación.



32 KB

Tamaño Total de la Caché.



64 Bytes

Tamaño Fijo por Bloque.



1 Ciclo

Costo Computacional por Acierto (Hit).
Representa el escenario ideal.



100 Ciclos

Costo Computacional por Fallo (Miss).
Representa la penalización extrema
por buscar en RAM.

Perfil de Carga: Patrones de Acceso Simulado

70% Accesos Secuenciales

Simula el recorrido de arrays o estructuras contiguas en memoria. El flujo más predecible.

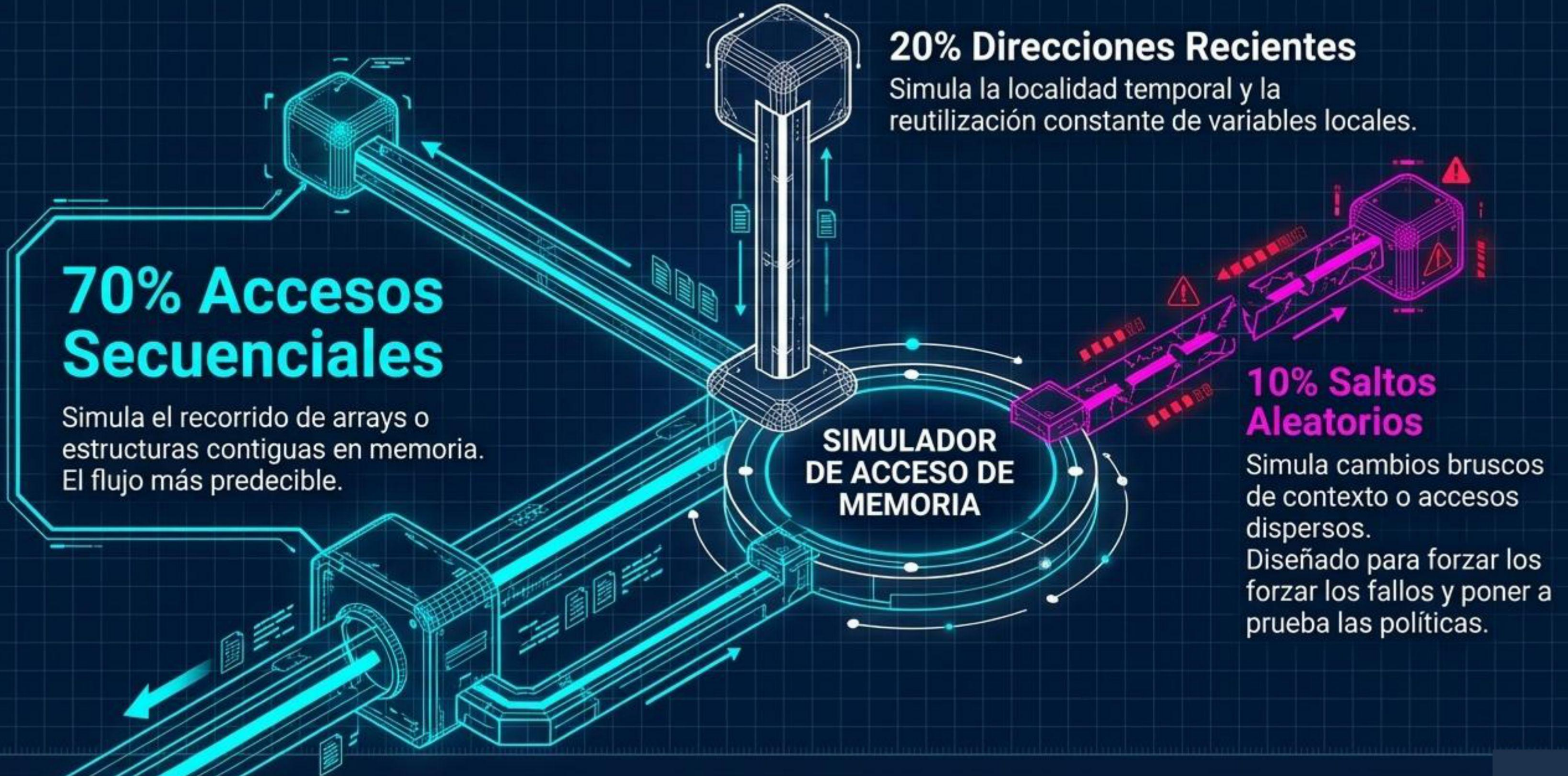
20% Direcciones Recientes

Simula la localidad temporal y la reutilización constante de variables locales.

10% Saltos Aleatorios

Simula cambios bruscos de contexto o accesos dispersos. Diseñado para forzar los fallos y poner a prueba las políticas.

**SIMULADOR
DE ACCESO DE
MEMORIA**



Cuantificación del Rendimiento (Métricas).

1. Hit Ratio (%).

95.50%

← Ejemplo Representativo (Ideal).

Definición: El porcentaje de accesos totales donde los datos solicitados ya residían en la caché. Es el indicador primario de éxito algorítmico.



2. Ciclos Teóricos de CPU.

(Total Hits × 1) + (Total Misses × 100)

Costo por Acierto

Costo por Fallo (Penalización)

Propósito: Traduce la precisión del algoritmo en un costo de tiempo de procesamiento simulado.



3. Simulación.

450ms

← Tiempo de Simulación

Propósito: Medición empírica del costo de sobrecarga computacional del propio simulador en la máquina anfitriona.

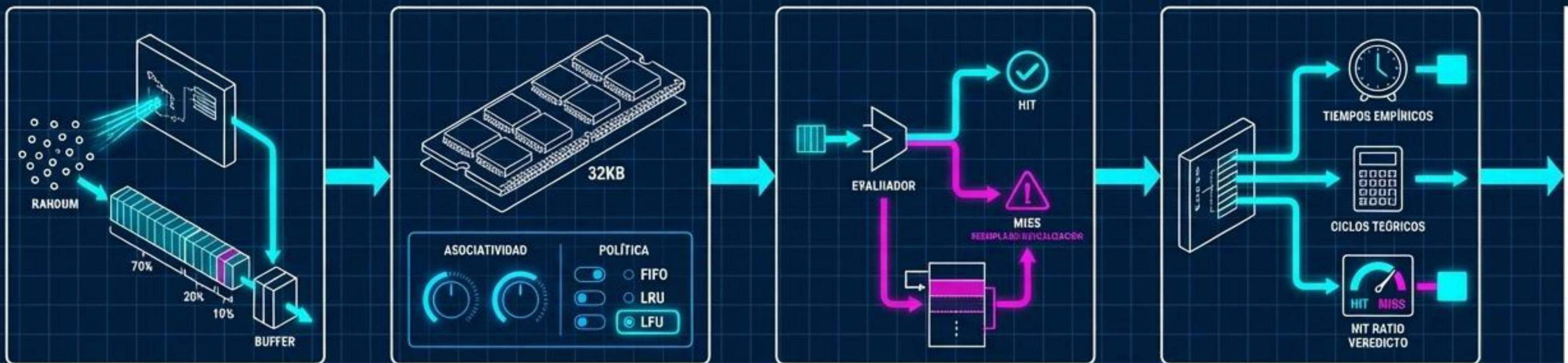


4. Tiempo Real de Ejecución.

Propósito: Medición empírica del costo de sobrecarga computacional del propio simulador en la máquina anfitriona.



Pipeline del Simulador de Caché



[Paso 1] Generación

Creación de la traza de memoria bajo la semilla probabilística (70/20/10).

[Paso 2] Configuración

Inicialización de la memoria (32KB), niveles de asociatividad y selección de la Política (FIFO, LRU o LRU).

[Paso 3] Procesamiento

Lectura secuencial de direcciones. Evaluación en tiempo real de Hit/Miss y ejecución de algoritmos de reemplazo o actualización.

[Paso 4] Resultados

Exportación de tiempos empíricos, cálculo de ciclos teóricos y veredicto comparativo del Hit Ratio.